Practicum No.14 **Date:…….**

GPU Execution – Subtraction & Division (CPU vs GPU)

I. Practical Significance

This experiment demonstrates the concept of **parallel computing using GPU** by performing **vector subtraction and division** operations on the NVIDIA Jetson Nano. It highlights the difference between **CPU's sequential execution** and **GPU's parallel processing**.
By comparing execution time using **NumPy (CPU)** and **CuPy (GPU)**, students understand how GPUs efficiently handle large datasets. The experiment also introduces **memory transfer between CPU and GPU**, and evaluates performance improvements in real-time embedded systems.

II. Competency and Industry-Specific Practical Skills

This practicum exercise is expected to develop the following skills for the industry-identified competency:

- **Apply** GPU computing principles to perform efficient subtraction and division operations using embedded systems. *(As stated in the Course Structure)*

a) **Identify** suitable computational tasks such as vector subtraction and division that can benefit from parallel execution using the NVIDIA Jetson Nano. *(PDO)*
b) **Implement** and compare CPU-based computation using NumPy and GPU-based computation using CuPy for subtraction and division operations. *(CDO)*
c) **Analyze** execution time and evaluate performance improvement achieved through GPU acceleration for different input sizes. *(PDO)*

III. Relevant CO(s) *(As stated in the Course Structure)*

- **CO1. *Evaluate*** GPU system architectures to enhance high-performance computing solutions.

IV. Practical Outcome (PrO) *(As stated in the Course Structure)*

- ***Develop*** instruction-level parallelism (ILP) techniques to efficiently perform **vector subtraction and division** for the given set of instructions.

V. Related ADO(s) *(As stated in the Course Structure)*

a) ***Follow*** safe computing and coding practices
b) ***Practice*** teamwork and documentation discipline
c) ***Comply*** with ethical coding conventions

VI. Minimum Underpinning Theory *(Include relevant content & images for this practicum.)*

GPU execution is based on **parallel processing**, where multiple computations are performed simultaneously to improve performance. Unlike a CPU, which executes instructions sequentially, a GPU contains many smaller cores that handle thousands of threads in parallel. The NVIDIA Jetson Nano is an embedded platform with an integrated GPU designed for accelerating compute-intensive tasks such as **vector subtraction, division operations, image processing, and AI applications**.

In GPU computing, tasks are divided into smaller sub-tasks and executed concurrently using a large number of threads. Libraries like **CuPy** enable GPU-based computation, while **NumPy** is used for CPU-based execution, allowing performance comparison.

In this experiment, **vector subtraction and division** are performed using both CPU and GPU. Since these operations are applied element-wise on large datasets, they benefit significantly from GPU acceleration due to parallel execution.

VII. Practicum Set-up/Circuit Diagram/Algorithm/Flowchart *(Include probable photos of experimental set-up.)*

Students must have:

- NVIDIA Jetson Nano with power supply and SD card
- JetPack SDK installed (Ubuntu OS with CUDA support)
- Python installed

- Required libraries: NumPy (CPU) and CuPy (GPU)
- Script file: **gpu_sub_div.py**
- Input dataset (vector size and random input arrays)
- Basic knowledge of Python programming and parallel computation

Algorithm

1. Start the program
2. Import required libraries (NumPy, CuPy, time)
3. Define input size
4. Generate random vectors A and B
5. Perform subtraction and division using CPU
6. Transfer data to GPU memory
7. Perform subtraction and division using GPU
8. Measure execution time for both
9. Display results
10. Stop the program

VIII. Resources Required

S. No.	Resource	Suggested Broad Specification	Quantity
1	NVIDIA Jetson Nano	Quad-core ARM CPU, 4GB RAM, integrated GPU, microSD storage	1 No.
2	Power Supply & Accessories	5V/4A power adapter, HDMI monitor, keyboard, mouse	1 Set.
3	JetPack SDK	Ubuntu OS with CUDA, cuDNN support	1 No.
4	Python	Python 3.x environment	1 No.
5	NumPy	CPU-based numerical computation library	1 No.
6	CuPy	GPU-accelerated array computation library	1 No.
7	Internet Connection	For installation of required libraries and updates	1 No.

IX. Safety Precautions

a) ***Save*** work frequently and maintain backup copies of your Python scripts.

b) ***Comment*** the code properly and include a header with author name, date, input size, and execution details.
c) ***Close*** unnecessary applications on the NVIDIA Jetson Nano to ensure sufficient memory and better GPU performance.
d) ***Validate*** input data (vector size, data type) to avoid runtime errors and memory overflow during CPU and GPU execution.

X. Procedure *(Self-Instructional)*

a) **Execute** both CPU and GPU operations for comparison
b) **Accept** the required input values such as vector size and input data (arrays A and B).
c) **Validate** the inputs for correct numeric format and ensure compatibility with the selected execution mode on the NVIDIA Jetson Nano.
d) **Perform** vector **subtraction and division** using the selected mode (CPU with NumPy or GPU with CuPy).
e) **Display** the computed output vectors and execution time clearly in the console.
f) **Allow** the user to repeat the execution with different input sizes or modes for comparison.
g) **Terminate** the program when the user chooses to exit.

Step 1: Boot Jetson Nano

- Insert SD card with JetPack SDK
- Power ON and complete initial setup

Step 2: Install Required Libraries

Open Terminal and type:

```
sudo apt update
pip3 install numpy
pip3 install cupy-cuda11x
```

Step 3: Create Python Program

Create a file:

```
nano gpu_sub_div.py
```

 # Ex No 14: Subtraction & Division (CPU vs GPU)

```
import numpy as np
import cupy as cp
import time

size = 10**6

a_cpu = np.random.rand(size)
b_cpu = np.random.rand(size) + 1 # avoid division by zero

# CPU
start = time.time()
sub_cpu = a_cpu - b_cpu
div_cpu = a_cpu / b_cpu
cpu_time = time.time() - start
```

```
# GPU
a_gpu = cp.array(a_cpu)
b_gpu = cp.array(b_cpu)

start = time.time()
sub_gpu = a_gpu - b_gpu
div_gpu = a_gpu / b_gpu
cp.cuda.Stream.null.synchronize()
gpu_time = time.time() - start

print("CPU Time:", cpu_time)
print("GPU Time:", gpu_time)
```

Step 4: Run the Program

python3 gpu_sub_div.py

Sample Input:

A = [1.2, 2.5, 3.1, 4.0, 5.3]
B = [0.8, 1.5, 2.9, 3.0, 4.7]

Expected Output:

Subtraction (CPU): [0.4, 1.0, 0.2, 1.0, 0.6]
Subtraction (GPU): [0.4, 1.0, 0.2, 1.0, 0.6]
Division (CPU): [1.5, 1.67, 1.07, 1.33, 1.13]
Division (GPU): [1.5, 1.67, 1.07, 1.33, 1.13]

CPU Time: 0.00050
GPU Time: 0.00032

XI. Actual Resources Used (*To be filled by the students*)

S. No.	Name of Resource	Broad Specifications		Quantity	Remarks (If any)
		Make	Details		
1.	NVIDIA Jetson Nano	NVIDIA	Quad-core ARM CPU, 4GB RAM, integrated GPU	1	Used for GPU execution
2.	Power Supply & Accessories	Generic	Generic 5V/4A adapter, HDMI monitor, keyboard, mouse	1 Set	Required for setup
3.	JetPack SDK	NVIDIA	Ubuntu OS with CUDA support	1	Pre-installed
4.	Python with NumPy & CuPy	Open Source	Python 3.x with required libraries	1	Used for implementation

XII. Actual Procedure Followed (*To be filled by the students*)

1. Booted the NVIDIA Jetson Nano and opened the Python environment.
2. Installed required libraries such as NumPy and CuPy using terminal commands.
3. Created a Python script (gpu_sub_div.py) for performing vector subtraction and division.
4. Defined the input size and generated random input vectors A and B.

5. Ensured that vector B values were adjusted to avoid division by zero errors.
6. Executed subtraction and division operations using CPU with NumPy.
7. Measured CPU execution time using the time module.
8. Transferred input data from CPU memory to GPU memory using CuPy arrays.
9. Executed subtraction and division operations using GPU (parallel processing).
10. Used synchronization to ensure GPU execution completion before measuring time.
11. Measured GPU execution time accurately.
12. Compared CPU and GPU outputs to verify correctness.
13. Repeated execution for consistency and observed variation in execution times.
14. Recorded and analyzed performance differences between CPU and GPU execution.

XIII. Observations and Dimensional Measurement *(To be filled by the students)*

The experiment was successfully performed on the NVIDIA Jetson Nano by executing **vector subtraction and division** operations using CPU with NumPy and GPU with CuPy. It was observed that for **smaller input sizes**, the execution time of CPU and GPU was nearly similar due to the overhead involved in transferring data from CPU memory to GPU memory. However, as the input size increased, the **GPU execution time became significantly lower than CPU execution time**, clearly demonstrating the advantages of **parallel processing**.

The subtraction and division results obtained from both CPU and GPU were **identical**, confirming the correctness and accuracy of GPU-based computation. It was also observed that **division operations are slightly more computationally intensive** compared to subtraction, which further highlights the benefit of GPU acceleration for complex arithmetic operations. Minor variations in execution time were noted due to system load, background processes, and memory usage on the Jetson Nano.

XIV. Results/Observations *(To be filled by the students)*

1. The subtraction and division results obtained using CPU (NumPy) and GPU (CuPy) were identical, confirming correctness of execution.
2. GPU execution showed significantly better performance for larger input sizes.
3. CPU execution time increased linearly with input size due to sequential processing.
4. GPU execution time increased at a slower rate due to parallel computation.
5. Data transfer overhead affected GPU performance for smaller datasets.
6. Division operations required more processing time compared to subtraction.
7. GPU demonstrated higher efficiency for compute-intensive operations.
8. Parallel execution reduced overall computation time.
9. Execution results remained consistent across multiple runs.
10. GPU performance advantage became more evident as data size increased.

XV. Interpretation of Results *(Students to state the meaning of the above-obtained results/observations)*

The results indicate that GPU-based execution on the NVIDIA Jetson Nano is highly efficient for performing large-scale subtraction and division operations due to its ability to process multiple data elements simultaneously using parallel architecture. While CPU performs operations sequentially, GPU divides the workload into smaller tasks and executes them concurrently, resulting in improved performance.

The identical outputs from CPU (NumPy) and GPU (CuPy) confirm the **accuracy and reliability of GPU computation**. The observed performance difference for smaller datasets highlights the **impact of data transfer overhead** between CPU and GPU memory. However, as the dataset size increases, this overhead becomes negligible, and GPU performance dominates. The experiment also demonstrates that **division operations benefit more from GPU acceleration** compared to simpler operations due to higher computational complexity.

Overall, the experiment emphasizes that GPU acceleration is highly beneficial for **data-intensive and real-time applications**, improving execution speed and computational efficiency.

XVI. Conclusions *(Students to draw conclusions (or take decisions) based on the interpretation of results*

The experiment successfully demonstrated the implementation of **vector subtraction and division using both CPU and GPU** on the NVIDIA Jetson Nano platform. The results confirmed that GPU-based execution using CuPy produces accurate outputs identical to CPU execution using NumPy, ensuring correctness and reliability of the computation.

It was observed that GPU execution significantly reduces computation time for large datasets due to its **parallel processing capability**, whereas CPU execution follows a sequential approach, resulting in comparatively higher execution time. The impact of data transfer overhead was noticeable for smaller datasets, but it became negligible for larger input sizes, where GPU performance showed a clear advantage.

In conclusion, GPU acceleration proves to be highly efficient for performing complex arithmetic operations such as subtraction and division in large-scale data processing. This makes GPU-based computation suitable for **high-performance computing, embedded systems, real-time applications, and data-intensive tasks**, thereby enhancing overall system performance and efficiency.

XVII. Suggested Practicum Related Questions:

Note: *Below are a few questions related to industry-specific higher-order thinking skills (HOT) that teachers must use to ensure students achieve the predetermined course outcomes.*

1. Implement vector subtraction using CPU with NumPy and GPU with CuPy, and compare their execution times on the NVIDIA Jetson Nano.

Implement vector operations using CPU (NumPy) and GPU (CuPy) and compare execution times

CODE:

```
import numpy as np
import cupy as cp
import time

size = 10**6

a = np.random.rand(size)
b = np.random.rand(size) + 1

# CPU
start = time.time()
sub_cpu = a - b
div_cpu = a / b
cpu_time = time.time() - start

# GPU
a_gpu = cp.array(a)
b_gpu = cp.array(b)

start = time.time()
sub_gpu = a_gpu - b_gpu
div_gpu = a_gpu / b_gpu
cp.cuda.Stream.null.synchronize()
gpu_time = time.time() - start

print("CPU Time:", cpu_time)
print("GPU Time:", gpu_time)
```

Theory

This implementation compares **CPU-based execution using NumPy** and **GPU-based execution using CuPy** for subtraction and division operations. The CPU processes operations sequentially, while the GPU executes them in parallel. As a result, GPU execution is faster for large datasets, demonstrating the advantage of **parallel computing in embedded systems**.

2. Analyze the performance improvement by varying input vector sizes and explain the impact of parallel processing on execution time.

Analyze performance improvement by varying input sizes

CODE:

```
import numpy as np
import cupy as cp
import time

sizes = [10**3, 10**5, 10**6]

for size in sizes:
    a = np.random.rand(size)
    b = np.random.rand(size) + 1

    # CPU
    start = time.time()
    a - b
    a / b
    cpu_time = time.time() - start

    # GPU
    a_gpu = cp.array(a)
    b_gpu = cp.array(b)

    start = time.time()
    a_gpu - b_gpu
    a_gpu / b_gpu
    cp.cuda.Stream.null.synchronize()
    gpu_time = time.time() - start

    print("Size:", size, "CPU:", cpu_time, "GPU:", gpu_time)
```

Theory

This experiment analyzes how performance changes with input size. For **small datasets**, CPU and GPU execution times are similar due to **data transfer overhead**. As the input size increases, GPU shows better performance because it can process multiple elements simultaneously. This demonstrates the **scalability of GPU computing**.

3. Demonstrate data transfer between CPU and GPU memory and evaluate how it affects overall performance in GPU execution.

Demonstrate data transfer between CPU and GPU memory and its impact

CODE:

```
import numpy as np
import cupy as cp
import time
```

```
size = 10**6

a = np.random.rand(size)

# Measure transfer time
start = time.time()
a_gpu = cp.array(a)
transfer_time = time.time() - start

print("Data Transfer Time (CPU to GPU):", transfer_time)
```

Theory

In GPU computing, data must be transferred from CPU memory to GPU memory before execution. This introduces **data transfer overhead**, which can reduce performance benefits for small datasets. However, for large datasets, the speed of parallel computation outweighs this overhead, making GPU execution more efficient overall.

Results may vary slightly depending on system load.
The experiment demonstrates that GPU execution provides better performance for large-scale subtraction and division operations, while data transfer overhead influences performance for smaller datasets.

XVIII References / Suggestions for further reading

a) NVIDIA Developer – Jetson Nano Documentation: https://developer.nvidia.com/embedded/jetson-nano-developer-kit
b) CUDA Toolkit Documentation: https://docs.nvidia.com/cuda/
CuPy Official Documentation: https://docs.cupy.dev/en/stable/
NumPy Official Documentation: https://numpy.org/doc/
c) GPU Computing Basics – YouTube – NVIDIA Developer
d) Parallel Computing with Python (NumPy vs CuPy) – YouTube Tutorials

XIX Suggested Assessment Scheme

The performance indicators given serve as a guideline for assessing the *'process-related skills/LOs'* (marks to be awarded in real-time in the laboratory by the faculty member, as these cannot be measured after the practicum is over) and *'product-related skills/LOs'*

Note: The weightage for the ***process-related skills*** and ***product-related skills*** will change depending on the PrO, COs, and the competency related to the concerned course.

Performance Indicators		Maximum Marks	Marks Obtained
Process-related Skills: 18 Marks - 60%			
1	Correct usage of Python and handling input data	6	
2	Correct implementation of CPU (NumPy) and GPU (CuPy) execution	5	
3	Safe practices, proper coding and documentation	4	
4	Team participation	3	
Product-related Skills: 12 Marks - 40%			
1	Accurate EA results	3	
2	Correct interpretation	2	
3	Quality of outputs (tables/plots)	3	
4	Conclusion quality	2	
5	HOT question responses	1	
6	Journal submission	1	
	Total	**30**	

To be filled by Student:

S.No.	Name of the Student	Register No.
1	**Neavis Rishva. J**	**URK25CS1236**

To be filled by the Faculty Member:

Marks Obtained			Name and Designation of the Faculty Member	Date of Evaluation
Process-Related Skills (18)	**Product-Related Skills (12)**	**Total (30)**		

******

ARISE
AND
SHINE
Karunya